
MANUAL DE USUARIO

para

Sistema de Lectura de Crotales de Animales

Version 1.0

Preparado por

Andrea Celeste Curcio, Jaume Adrover

Jaudre CV Services
Campus de Móstoles
Calle Tulipán s/n
28933 Móstoles, Madrid

30 de abril de 2026

Historial de versiones

Nombre	Fecha	Cambios	Version
Jaume Adrover	2026-04-23	Versión inicial del documento	1.0

Índice general

1	Introducción	3
2	Guía del instalador	4
2.1	Requisitos del sistema	4
2.2	Acceso a la Raspberry Pi	4
2.3	Instalación de dependencias	5
2.4	Clonación del repositorio	6
2.5	Preparación del directorio de datos	6
2.6	Construcción y ejecución con Docker	6
2.7	Ejecución alternativa sin Docker	7
2.8	Notas importantes	8
2.9	Despliegue del sistema	9
2.9.1	Despliegue mínimo	9
2.9.2	Despliegue de desarrollo	9
2.9.3	Despliegue máximo	9
2.10	Arquitectura de despliegue	9
3	Guía del usuario	11
3.1	Ejecución sobre una imagen	11
3.2	Procesamiento por lotes (batch)	12
3.3	Notas importantes	13
3.4	Problemas comunes	13

1 Introducción

Este proyecto implementa una herramienta para la lectura automática de crotales bovinos mediante técnicas de visión artificial y reconocimiento óptico de caracteres (OCR). Su objetivo es detectar y extraer el identificador numérico presente en los crotales a partir de imágenes.

El código fuente del proyecto se encuentra disponible en el siguiente repositorio:

https://github.com/jaaumeadrover/AIVA_2026-BovineVision

¿Qué es esta herramienta?

Se trata de una aplicación que procesa imágenes de crotales y extrae automáticamente el número identificador mediante un flujo de preprocesamiento y OCR. Puede trabajar tanto con imágenes individuales como con conjuntos de imágenes (modo batch).

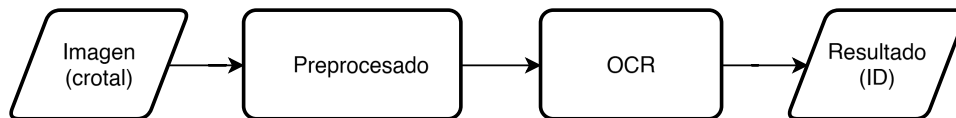


Figura 1: Flujo de procesamiento del sistema OCR

¿Para qué sirve?

Permite automatizar la lectura de identificadores de animales, evitando la intervención manual. Esto facilita tareas como:

- Procesar grandes cantidades de imágenes de forma automática
- Reducir errores en la lectura de identificadores
- Generar resultados estructurados (archivos de salida)

Utilidad para el usuario

El usuario puede ejecutar la aplicación sobre imágenes de crotales y obtener el identificador detectado. Además, es posible validar los resultados frente a un conjunto de datos de referencia (dataset) y detectar posibles duplicados en el procesamiento por lotes.

El uso de la herramienta se describe paso a paso en las secciones siguientes de este manual.

2 Guía del instalador

En esta sección se describe paso a paso cómo instalar la aplicación y ponerla en funcionamiento en la Raspberry Pi.

Nota: Aunque el sistema está diseñado para ejecutarse en una Raspberry Pi, también puede probarse en un ordenador personal con Python y las dependencias instaladas, lo que permite validar su funcionamiento antes del despliegue final.

2.1. Requisitos del sistema

Para el correcto funcionamiento del sistema se requiere:

- Raspberry Pi 5
- Sistema operativo Raspberry Pi OS (64 bits)
- Conexión a red local
- Acceso a terminal (local o mediante SSH)
- Conexión a Internet (para instalación de dependencias)
- Fuente de alimentación adecuada (USB-C, 5V / 5A recomendada)

Opcional (según el modo de ejecución seleccionado):

- Docker: recomendado para despliegue en Raspberry Pi, ya que encapsula todas las dependencias.
- Python 3.12: necesario únicamente para la ejecución en local sin Docker.

2.2. Acceso a la Raspberry Pi

Antes de comenzar, es necesario acceder a la Raspberry Pi donde se ejecutará el sistema.

Opción A: Acceso remoto (recomendado)

Desde otro ordenador, abrir una terminal y conectarse mediante SSH:

```
ssh alumno@raspberrypi.local
```

Al conectarse, se solicitará la contraseña del usuario configurada en la Raspberry Pi.

Notas:

- Si es la primera conexión, puede aparecer un aviso de seguridad. En ese caso, escribir **yes** y pulsar Enter.
- La Raspberry Pi debe estar conectada a una red (WiFi o Ethernet) para poder descargar el repositorio e instalar dependencias.

Si la Raspberry Pi no dispone de conexión a red, es necesario configurar el acceso WiFi mediante:

```
sudo raspi-config
```

Seleccionar *System Options* → *Wireless LAN* e introducir las credenciales de la red.

Opción B: Acceso directo

Si la Raspberry dispone de pantalla y teclado, abrir directamente la aplicación *Terminal*.

2.3. Instalación de dependencias

Git

Comprobar si Git está instalado:

```
git --version
```

Si no está instalado:

```
sudo apt update  
sudo apt install git
```

Docker

Comprobar si Docker está instalado:

```
docker --version
```

Si no está disponible, instalar Docker:

```
curl -fsSL https://get.docker.com | sh
```

Configurar permisos (recomendado):

```
sudo usermod -aG docker $USER
```

Tras este paso, los comandos de Docker podrán ejecutarse sin necesidad de utilizar **sudo**. Cerrar sesión y volver a iniciarla.

Nota: Si Docker requiere permisos, los comandos pueden ejecutarse utilizando **sudo**.

2.4. Clonación del repositorio

Una vez conectado a la Raspberry Pi (ya sea de forma local o mediante SSH), ejecutar en la terminal:

```
git clone https://github.com/jaaumeadrover/AIVA_2026-BovineVision
```

A continuación, acceder al directorio del proyecto:

```
cd AIVA_2026-BovineVision
```

2.5. Preparación del directorio de datos

El sistema utiliza una carpeta llamada **data** para almacenar las imágenes de entrada y los resultados generados.

Esta carpeta ya está incluida en el repositorio. En ella se encuentra una imagen de ejemplo (0001.TIF) que puede utilizarse para verificar el funcionamiento del sistema.

El usuario puede añadir nuevas imágenes dentro de esta carpeta para su procesamiento.

Esta carpeta se utiliza además para intercambiar datos entre el sistema host y el contenedor Docker.

Ejemplo de uso:

- Entrada: data/imagen.TIF
- Salida: data/resultado.txt

2.6. Construcción y ejecución con Docker

Construcción de la imagen

La construcción de la imagen consiste en generar un entorno de ejecución que incluye todas las dependencias y el código del sistema a partir del archivo **Dockerfile**.

Dentro del directorio del proyecto:

```
docker build -t myapp .
```

Nota: En algunos sistemas es necesario ejecutar los comandos de Docker con privilegios de administrador. En ese caso, se debe anteponer **sudo** al comando. Por ejemplo:

```
sudo docker build -t myapp .
```

Este proceso puede tardar varios minutos.

Ejecución del contenedor

```
docker run -it -v $(pwd)/data:/app/data myapp
```

Este comando:

- Inicia el contenedor en modo interactivo
- Vincula la carpeta `data` con el contenedor

Una vez iniciado, se abrirá una terminal dentro del contenedor.

Desde ahí se pueden ejecutar los scripts del sistema, por ejemplo:

```
python -m src.predict --img data/0001.TIF --out data/resultado.txt
```

En la Figura 2 se muestra un ejemplo de ejecución del sistema sobre una imagen de prueba:

```
(base) alumno@raspberrypi:~/AIVA_2026-BovineVision $ sudo docker run -it -v $(pwd)/data:/app/data myapp
(my_env) appuser@9a270ff3aad9:/app$ python -m src.predict --img data/0001.TIF --out data/resultado.txt
--- Iniciando predict.py ---
Modo: Imagen individual -> data/0001.TIF
--- Inicializando EasyOCR Engine (Cargando modelos...) ---
Using CPU. Note: This module is much faster with a GPU.
Downloading detection model, please wait. This may take several minutes depending upon your network connection.
Progress: | 100.0% Complete
Downloading recognition model, please wait. This may take several minutes depending upon your network connection.
Progress: | 100.0% Complete
--- Procesando imagen: 0001.TIF ---
/home/appuser/.conda/envs/my_env/lib/python3.12/site-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
  super().__init__(loader)
Resultado: 0288 (15.28s)
(my_env) appuser@9a270ff3aad9:/app$
```

Figura 2: Ejecución del sistema OCR en contenedor Docker sobre la imagen de prueba.

2.7. Ejecución alternativa sin Docker

Como alternativa al despliegue mediante Docker, el sistema puede ejecutarse directamente en el ordenador, lo que facilita tareas de desarrollo y pruebas.

En este caso, los comandos se ejecutan desde la terminal del sistema (fuera de Docker).

1. Crear entorno virtual (opcional)

```
python -m venv venv
source venv/bin/activate % Linux / Mac
venv\Scripts\activate % Windows
```

2. Instalar dependencias

```
pip install -r requirements.txt
```


3. Preparar la carpeta de datos

El repositorio incluye una carpeta **data** destinada a almacenar las imágenes de entrada y los resultados generados.

En caso de no existir (por ejemplo, si se ha eliminado), puede crearse manualmente:

```
mkdir data
```

4. Ejecutar prueba

El repositorio incluye una imagen de ejemplo en **data/0001.TIF**, que puede utilizarse para comprobar el funcionamiento del sistema.

```
python -m src.predict --img data/0001.TIF --out data/resultado.txt
```

Nota: El usuario puede sustituir esta imagen por cualquier otra imagen de crotal colocándola en la carpeta **data**.

Si todo funciona correctamente, se generará un archivo **resultado.txt** en la carpeta **data** con el identificador detectado.

2.8. Notas importantes

General

- La carpeta **data** es obligatoria para la entrada y salida de datos.
- Asegúrese de que las rutas de entrada y salida son correctas.
- El sistema puede ejecutarse tanto con Docker como directamente en local.
- Las imágenes de entrada deben colocarse en la carpeta **data**.
- Los comandos deben ejecutarse desde la raíz del proyecto para garantizar el correcto funcionamiento del sistema.
- Los nombres de archivo distinguen entre mayúsculas y minúsculas, por lo que la ruta debe coincidir exactamente con el nombre del archivo (por ejemplo, **0001.TIF** no es lo mismo que **0001.tif**).

Ejecución con Docker

- El flag **-it** permite interactuar con la terminal del contenedor.
- El volumen **-v \$(pwd)/data:/app/data** permite compartir archivos entre el sistema y el contenedor.
- **\$(pwd)** hace referencia al directorio actual desde el que se ejecuta el comando.
- En ejecución con Docker, pueden requerirse permisos de escritura en la carpeta **data**. En caso de error, verificar los permisos del sistema de archivos.

2.9. Despliegue del sistema

En esta sección se describen los distintos escenarios de despliegue del sistema en función del entorno de ejecución.

2.9.1. Despliegue mínimo

El despliegue mínimo consiste en ejecutar el sistema en una Raspberry Pi utilizando Docker. Este escenario corresponde al flujo descrito previamente en la sección de construcción y ejecución con Docker.

Incluye los siguientes pasos:

- Clonación del repositorio
- Construcción de la imagen Docker
- Ejecución del contenedor

Este despliegue permite ejecutar el sistema de forma autónoma en el dispositivo objetivo.

2.9.2. Despliegue de desarrollo

Como alternativa al despliegue mediante Docker, el sistema puede ejecutarse en un ordenador personal sin utilizar contenedores, con el objetivo de facilitar pruebas y desarrollo.

Incluye:

- Creación de entorno virtual
- Instalación de dependencias
- Ejecución directa del script

2.9.3. Despliegue máximo

El despliegue máximo incluye la integración con un sistema externo de captura de imágenes (Data publisher), encargado de enviar automáticamente imágenes a la Raspberry Pi para su procesamiento.

Este escenario permite automatizar completamente el flujo de adquisición y procesamiento de datos.

2.10. Arquitectura de despliegue

A continuación, se describe la arquitectura de despliegue del sistema en su entorno objetivo.

El sistema se ejecuta sobre una Raspberry Pi 5 con sistema operativo Raspberry Pi OS (64 bits). En este dispositivo se utiliza un contenedor Docker que encapsula la aplicación y sus dependencias.

Dentro del contenedor se incluyen librerías como EasyOCR, OpenCV y NumPy, junto con el script principal `predict.py`, encargado del procesamiento de las imágenes.

El sistema utiliza una carpeta compartida (**data**) que permite intercambiar archivos entre la Raspberry Pi y el contenedor.

El sistema no incluye un mecanismo de captura automática de imágenes. Las imágenes de entrada deben ser proporcionadas externamente, ya sea mediante copia manual o transferencia desde otro dispositivo.

En este contexto, un dispositivo externo (denominado *Data publisher*) puede encargarse de enviar imágenes a la Raspberry Pi a través de la red local (por ejemplo, mediante SFTP). Este componente no forma parte del sistema principal, pero permite automatizar la entrada de datos.

Esta arquitectura permite desacoplar el procesamiento del origen de los datos, facilitando la integración con distintos sistemas de captura y mejorando la escalabilidad y el mantenimiento del sistema.

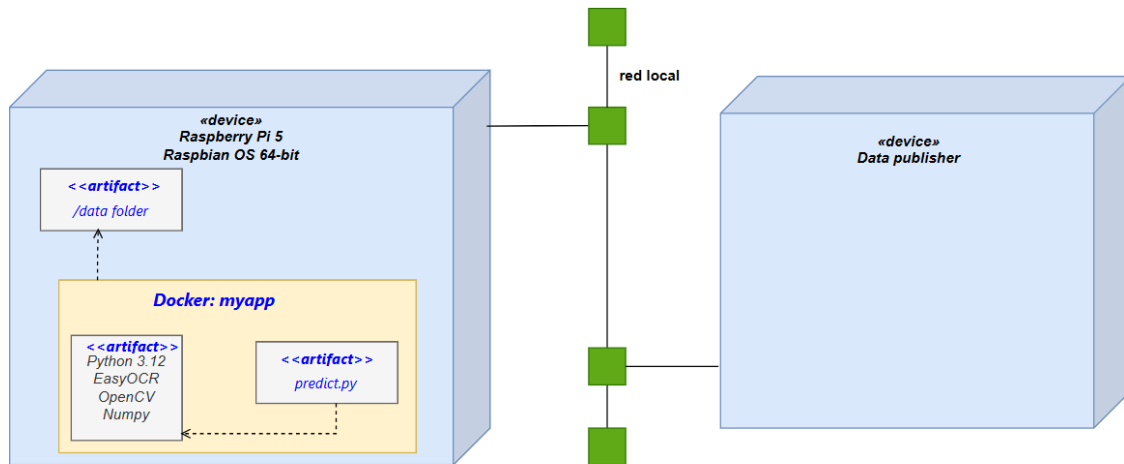


Figura 3: Arquitectura de despliegue del sistema

3 Guía del usuario

En esta sección se describe cómo utilizar el sistema una vez instalado.
Se asume que la instalación se ha realizado correctamente siguiendo la Guía del instalador.
Los comandos pueden ejecutarse:

- Dentro del contenedor Docker
- O directamente en el sistema local (ejecución con Python)

3.1. Ejecución sobre una imagen

1. Preparar la imagen de entrada

Copiar la imagen que se desea procesar en la carpeta `data` del proyecto.



Figura 4: Imagen de ejemplo de un crotal utilizada como entrada del sistema.

2. Ejecutar el sistema OCR

Ejecutar el siguiente comando:

```
python -m src.predict --img data/0001.TIF --out data/resultado.txt
```

Este comando procesa la imagen de entrada (Figura 4) y genera un archivo de salida en la carpeta `data`, como se muestra en la Figura 5.

 0001.TIF	30/05/2016 1:27	TIFImage.Document
 resultado.txt	26/04/2026 19:42	Documento de tex...

Figura 5: Contenido de la carpeta `data` tras la ejecución del sistema, incluyendo la imagen de entrada y el archivo de salida generado.

En la Figura 6 se muestra un ejemplo de la salida del sistema durante la ejecución, donde se observa el identificador detectado.

```
Progress: | 100.0% Complete --- Procesando imagen: 0001.TIF ---  
C:\Users\andre\AIVA_2026-BovineVision\venv\Lib\site-packages\torch\utils\data\loader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.  
  super().__init__(loader)  
Resultado: 0288 (168.48s)
```

Figura 6: Salida en terminal del sistema durante la ejecución, donde se muestra el identificador detectado correctamente (0288).

Nota: El archivo 0001.TIF es un ejemplo incluido en el repositorio. Puede utilizarse cualquier otra imagen colocada en la carpeta **data**, cambiando su nombre en el comando. El nombre del archivo de salida también puede modificarse.

Por ejemplo:

```
python -m src.predict --img data/nuevaimagen.TIF --out data/nuevaimagen.txt
```

3. Verificar el resultado

Para visualizar el resultado en la terminal:

```
cat data/resultado.txt
```

También se puede abrir el archivo **data/resultado.txt** con un editor de texto. El contenido del archivo puede observarse en la Figura 7, donde se muestra el identificador detectado correctamente.

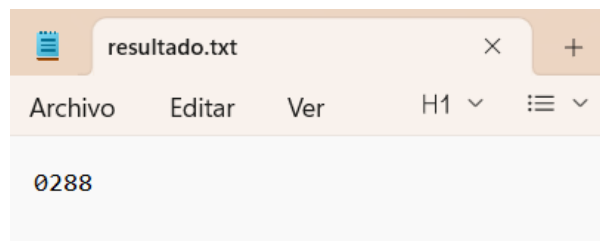


Figura 7: Contenido del archivo **resultado.txt**, donde el sistema detecta correctamente el identificador 0288.

3.2. Procesamiento por lotes (batch)

El sistema permite procesar múltiples imágenes de forma automática.

1. Preparar las imágenes

Copiar todas las imágenes a procesar dentro de la carpeta **data**.

2. Ejecutar en modo batch

```
python -m src.predict --dir data --eval data/resultados.csv
```

Este comando procesa todas las imágenes contenidas en la carpeta **data** y genera un archivo **resultados.csv** con los resultados.

3. Ver resultados

Para visualizar el resultado en la terminal:

```
cat data/resultados.csv
```

También se puede abrir el archivo **data/resultados.csv** con un editor de texto o una hoja de cálculo.

El archivo generado contiene:

- Nombre de la imagen
- Resultado OCR
- Validación (si aplica)
- Tiempo de procesamiento

3.3. Notas importantes

- Los comandos deben ejecutarse desde la raíz del proyecto.
- Todas las imágenes deben estar dentro de la carpeta **data**.
- Los resultados se guardan automáticamente en esta carpeta.
- Verificar que los nombres de archivo coinciden exactamente (mayúsculas/minúsculas).
- Si se utiliza Docker, los comandos deben ejecutarse dentro del contenedor en ejecución.

3.4. Problemas comunes

- **Error: No se encuentra la imagen**

Verificar que:

- La carpeta **data** existe
- La imagen está dentro de **data**
- El nombre del archivo coincide exactamente (incluyendo mayúsculas/minúsculas)

- **Error al ejecutar Docker**

Verificar que:

- Docker está instalado correctamente

- El servicio Docker está en ejecución
- El usuario tiene permisos (o ejecutar con **sudo**)

■ **No se genera archivo de salida**

Verificar que:

- La ruta de salida es correcta
- El script se ha ejecutado sin errores

■ **Error: Permission denied al escribir en data**

Este error puede aparecer al ejecutar el sistema dentro de Docker si el contenedor no tiene permisos de escritura sobre la carpeta compartida.

Para solucionarlo, ejecutar en la terminal:

```
sudo chmod -R 777 data
```

Esto otorga permisos de lectura y escritura a todos los usuarios sobre la carpeta **data**.

Nota: Este comando se utiliza como solución rápida en entornos de prueba.